

Case Studies

This chapter describes two *Jason* applications which are more elaborate than the examples presented in previous chapters. The purpose in presenting these applications is to demonstrate how *Jason* was used to develop solutions in those contexts.

The first application consists of a team of agents that took part in the second CLIMA Contest, held with CLIMA VII in 2006 [17, 33]. The main feature of this application is to integrate AgentSpeak and Java code, specially in order to use legacy code from within the agents and to interact with the contest simulation server. Even though the implementation was not completely finished in time for the contest, the team won the competition, and the experience also allowed us to improve various aspects of the interpreter. The version described here has some more improvements in the AgentSpeak code in comparison to the one used in the competition.

The second application is the implementation of the Electronic Bookstore designed as a case study in the ‘Developing Intelligent Agent Systems’ book by Padgham and Winikoff [75] using the Prometheus methodology. In the book, the system was implemented in JACK and we present here, in Section 9.2, its implementation using *Jason* instead. Although the main core of the application logic is developed in AgentSpeak, a Web-based interface, distributed agents and database access are also required in this application. Integration with these technologies can be useful for many applications of interest to readers of this book, therefore the description of this application here focuses on this integration, as the AgentSpeak code is rather simple. It is also useful for showing how *Jason* can be applied directly in the implementation of a system designed using a BDI-based agent-oriented software engineering methodology.

9.1 Case Study I: Gold Miners

We quote below the general description of the CLIMA contest's scenario; Figure 9.1 shows a screenshot.

Recently, rumours about the discovery of gold scattered around deep Carpathian woods made their way into the public. Consequently hordes of gold miners are pouring into the area in the hope to collect as much of gold nuggets as possible. Two small teams of gold miners find themselves exploring the same area, avoiding trees and bushes and competing for the gold nuggets spread around the woods. The gold miners of each team coordinate their actions in order to collect as much gold as they can and to deliver it to the trading agent located in a depot where the gold is safely stored.

(<http://cig.in.tu-clausthal.de/CLIMAContest/>)

To implement our team, two features of *Jason* were specially useful: architecture customisation and internal actions (see Figure 9.2). A customisation of the overall architecture, as explained in Section 7.3, is used to interface between the agent and its environment. The environment for the CLIMA contest was implemented by

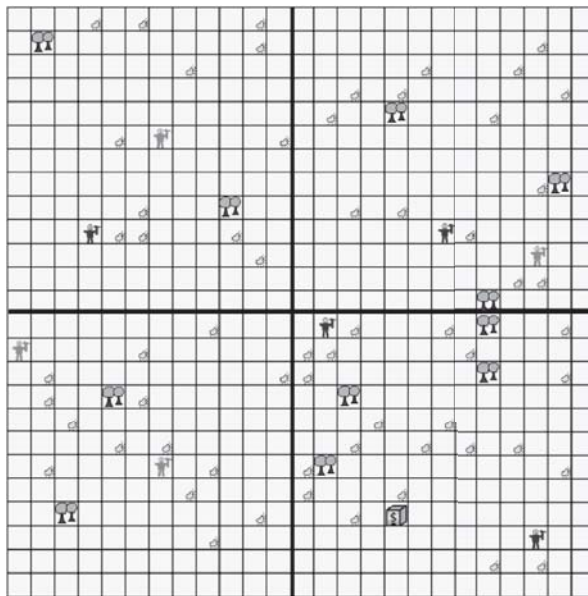


Figure 9.1 The contest scenario.

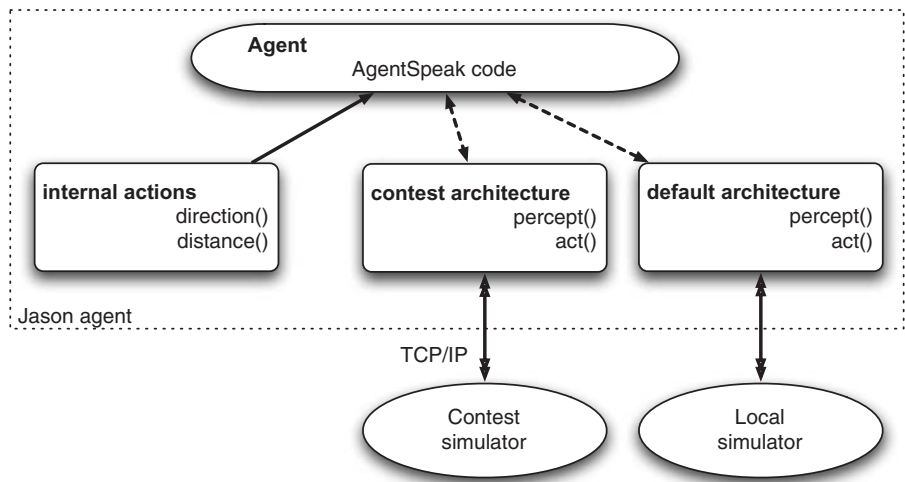


Figure 9.2 Agent extensibility and customisation.

the contest organisers in a remote server that simulates the mining field, sending perception to the agents and receiving requests for action execution according to a particular protocol also defined by the contest organisers. Therefore, when an agent attempts to perceive the environment, the architecture sends to it the information provided by the simulator, and when the agent chooses an action to be performed, the architecture sends the action execution request back to the simulator. For example, a plan such as:

```
+pos(X,Y) : Y > 0 <- left.
```

would be triggered when the agent perceives its position and its current line in the world grid is greater than zero. The `+pos(X,Y)` percept is produced by the architecture from the messages sent by the simulator, and `left` is an action that the architecture sends to the simulator for execution.

The architecture customisation was also very useful to bind the agent to an environment implementation, which could be either the real contest simulator or our own local simulator, by simply changing the project configuration. Since the contest simulator is not always available and the communication to/from it is quite slow, having a local simulator to develop and test the team was quite useful.

Although most of the agent is coded in AgentSpeak, some parts were implemented in Java, in this case because we wanted to use legacy code; in particular, we already had a Java implementation of the A* search algorithm, which we used to find paths in instances of the simulated scenario (it is interesting to note that in

one of the maze-like scenarios used in the competition, our team was the only one to successfully find a path to the depot). This algorithm was made accessible to the agents by means of internal actions (refer to Chapter 7.1, which explains how new internal actions can be created). The code below exemplifies how these internal actions might be used in AgentSpeak plans.

```
+gold(X,Y)
:   pos(X,Y) & depot(DX,DY) & carrying_gold
<- pick;
   jia.get_direction(X,Y,DX,DY,Dir);
   Dir.
```

In this plan, when the agent perceives some gold in its current position, it picks up the gold and calls the `get_direction` internal action of the `jia` library. This internal action receives two locations as parameters ($\langle X,Y \rangle$ and $\langle DX,DY \rangle$), computes a path between them using A* (using the Manhattan distance as heuristic, as usual in scenarios such as this), and unifies `Dir` with the first action (up, down, left, or right) according to the path found from the first towards the second coordinate. The plan then asks for the execution of the action instantiated to variable `Dir`. Note that this plan is illustrative; it does not generate the behaviour of carrying the gold to the depot; only one step towards it is performed in the excerpt above. Since each agent has only a partial view of the environment and some kind of information is useful for all of them, especially the location of obstacles for A*, the customised architecture ensures that such information was shared among all agents of the *Jason* team.

Overall Team Strategy

The team is composed of two roles played by five agents. The miner role is played by four agents which will have the goal of finding pieces of gold and carrying them to the depot. The team also has one other agent enacting the leader role; its goals are to allocate agents to the quadrants closest to their initial position in the scenario and to allocate free agents to a piece of gold that has been found by any of the team members. Note that exactly four (mining) agents need to log in to the simulation server for each team. The diagrams in Figures 9.3 – 9.5 give an overview of the system and those two roles using the Prometheus methodology [75].

The overall strategy of the team is as follows. Each miner is responsible for systematically (rather than randomly) searching for gold *within* one quadrant of the environment (see Figure 9.6). Since the initial positions of the agents are only known when the game starts, the allocation of the agents to quadrants depends on such positions. The team uses the protocol in Figure 9.7 for quadrant allocation. At the beginning of each game, the four mining agents send their location to the

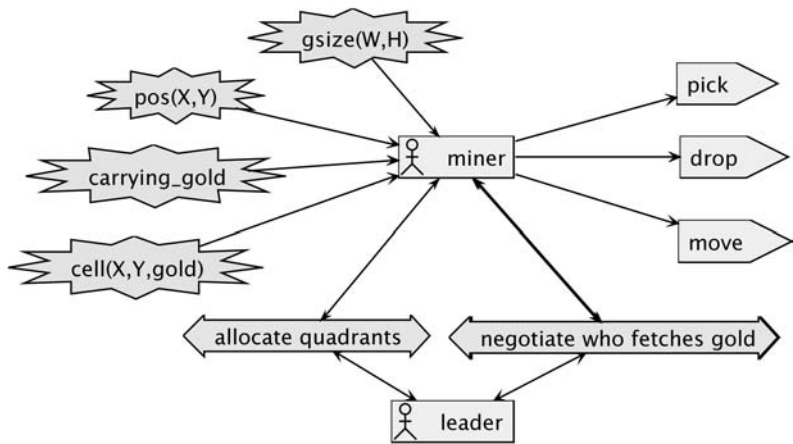


Figure 9.3 System overview diagram.

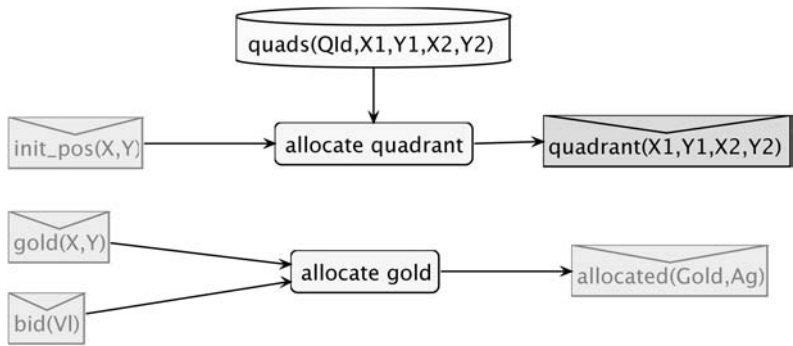


Figure 9.4 Leader agent overview diagram.

leader agent; the leader allocates each quadrant to an agent by checking which agent is nearest to that quadrant, and sends a message to each agent saying to which quadrant they have been allocated. We have decided to centralise some decisions in a leader agent so as to decrease the number of required messages in such distributed negotiations; even though all agents were run in the same machine in the actual competition, this is particularly important if in a future implementation we decide to run the agents in different machines, in case agents become too computationally heavy to run them all in one machine.

Another protocol is used to decide which agent will commit to retrieve a piece of gold found by a miner that is already carrying gold (Figure 9.8). When an agent (e.g. some miner in Figure 9.8) sees a piece of gold and cannot pick it up

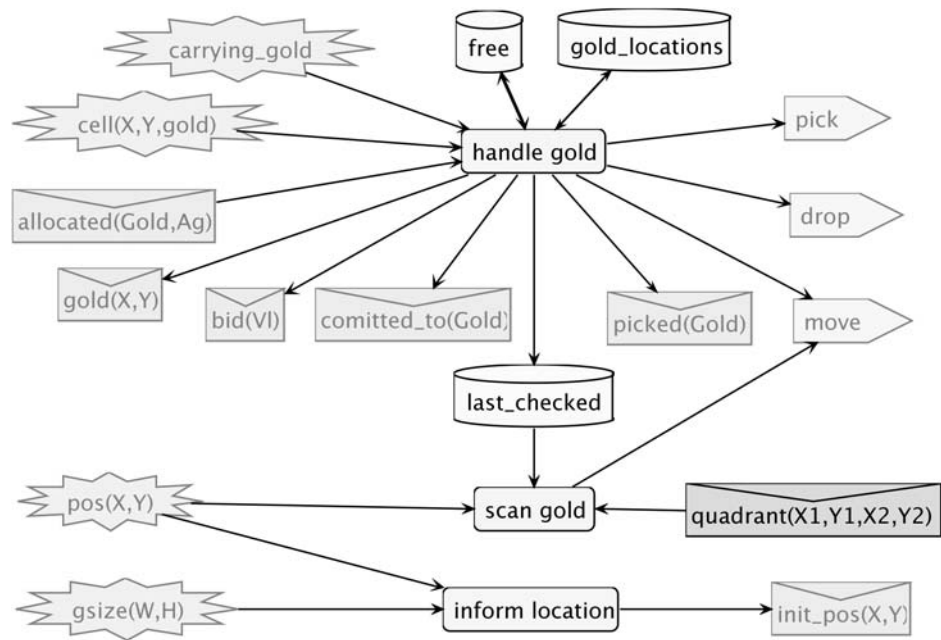


Figure 9.5 Miner agent overview diagram.

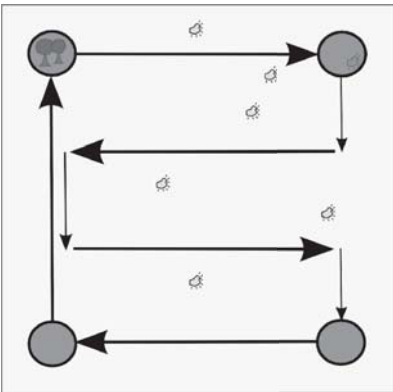


Figure 9.6 Miner's search path within its quadrant.

(e.g. because it is already carrying gold), it broadcasts the location of the piece of gold just found to all agents. The other agents bid to take on the task of fetching that piece of gold; such a bid is computed based on its availability and distance to that gold. All bids are sent to the leader who then chooses the best agent to commit to collect that piece of gold. This protocol also states that, whenever an agent

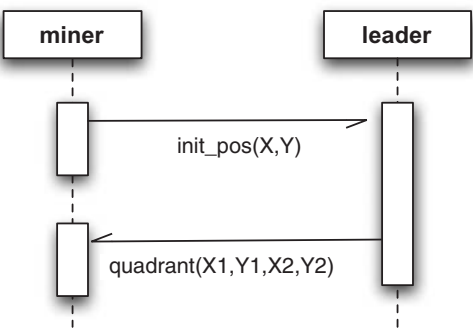


Figure 9.7 Quadrant allocation protocol.

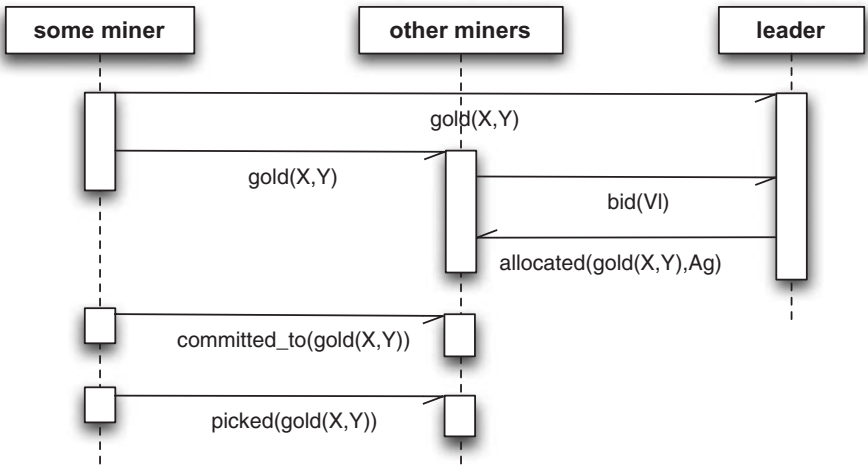


Figure 9.8 Gold allocation protocol.

decides to go towards some piece of gold or picks one, it should announce that fact to all other agents. Another agent that happens to also have the intention of fetching that same piece of gold can therefore drop the intention.

The miner agents have two mutually exclusive goals: ‘find gold’ and ‘handle gold’. Whenever the agent has currently no other intention, it adopts the goal of exploring its own quadrant to find gold. When the agent either perceives some gold or is allocated to a piece of gold following the protocol in Figure 9.8, it gives up the ‘find gold’ goal and commits to the goal of handling that particular piece of gold. When this latter goal is achieved, the agent commits again to the ‘find gold’ goal.

Below, some important excerpts of the AgentSpeak code are presented; the complete code and the local simulator of the contest are available in the *Jason* distribution (in folder `examples/Gold-Miners`).

Implementation of Quadrant Allocation

An agent may participate in many simulations, each one with a particular scenario defining specific obstacles, grid size and depot location. When a new simulation starts, the architecture adds a belief with the simulation identification (represented by variable *s* in the sources) and the grid size of the scenario. Once they have this information, the miners wait to be informed of their initial location in that scenario and send this information to the leader:

```
+gsize(S,_,_) : true // S is the simulation round ID
  <- !send_init_pos(S).
+!send_init_pos(S) : pos(X,Y)
  <- .send(leader,tell,init_pos(S,X,Y)).
+!send_init_pos(S) : not pos(,_ _) // if I do not know my position yet
  <- .wait("+pos(X,Y)", 500); // wait for it and try again
  !!send_init_pos(S).
```

The leader allocates the miners quadrants based on their positions on the grid:

```
@quads[atomic]
+gsize(S,W,H)
  <- // calculate the area of each quadrant and remember them
  +quad(S,1, 0, 0, W div 2 - 1, H div 2 - 1);
  +quad(S,2, W div 2, 0, W-1, H div 2 - 1);
  +quad(S,3, 0, H div 2, W div 2 - 1, H - 1);
  +quad(S,4, W div 2, H div 2, W - 1, H - 1).

+init_pos(S,X,Y)[source(A)]
  : // if all miners have sent their position
  .count(init_pos(S,_,_),4)
  <- // remember who doesn't have a quadrant allocated
  // (initially all miners)
  +~quad(S,miner1); +~quad(S,miner2);
  +~quad(S,miner3); +~quad(S,miner4);
  !assign_all_quads(S,[1,2,3,4]).

+!assign_all_quads(_,[]).
+!assign_all_quads(S,[Q|T])
  <- !assign_quad(S,Q);
  !assign_all_quads(S,T).
```



```
// assign quadrant Q to a miner
+!assign_quad(S,Q)
  : quad(S,Q,X1,Y1,X2,Y2) &
    ~quad(S,_) // there still is a miner without quadrant
  <- .findall(Ag, ~quad(S,Ag), LAgs);
    !calc_ag_dist(S,Q,LAgs,LD);
    .min(LD,d(Dist,Ag));
    .print(Ag, "'s Quadrant is: ",Q);
  --~quad(S,Ag);
    .send(Ag,tell,quadrant(X1,Y1,X2,Y2)).

+!calc_ag_dist(S,Q,[],[]).
+!calc_ag_dist(S,Q,[Ag|RAg],[d(Dist,Ag)|RDist])
  : quad(S,Q,X1,Y1,X2,Y2) & init_pos(S,AgX,AgY)[source(Ag)]
  <- // get the distance between X1,Y1 and AgX,AgY
    jia.dist(X1,Y1,AgX,AgY,Dist);
    !calc_ag_dist(S,Q,RAg,RDist).
```

Implementation of Gold Searching

When the miner is free,¹ it has to systematically search for pieces of gold in its quadrant; it ‘scans’ its quadrant as illustrated in Figure 9.6. The plans for achieving this goal determine that the agent should start from the place where it last stopped searching for gold (represented by the belief `last_checked(X,Y)`), or if there is no information about previous location where the search stopped then the agent starts from the upper-left position within its quadrant (represented by the belief `quadrant(X1,Y1,X2,Y2)[source(leader)]`). The obstacles may prevent the agent from being in the exact location defined by the scan strategy, so the agent can skip to a close location, provided it is ‘around’ the target location. As the agent can perceive gold in all neighbouring cells, it can skip two lines when moving vertically.

```
/* beliefs and rules */

last_dir(null). // the last movement I did
free.

// next line is the bottom of the quadrant,
// if 2 lines below is too far
calc_new_y(AgY,QuadY2,QuadY2) :- AgY+2 > QuadY2.

// otherwise, the next line is 2 lines below
calc_new_y(AgY,_,Y) :- Y = AgY+2.
```

¹The agent maintains a belief stating whether it is currently ‘free’ or not. Being free means that the agent is not committed to handling any piece of gold.

```

/* plans */

+free : last_checked(X,Y)      <- !around(X,Y).
+free : quadrant(X1,Y1,X2,Y2) <- !around(X1,Y1).
+free : true                   <- !wait_for_quad.

+!wait_for_quad : free & quadrant(_,_,_,_)
  <- -+free. // generate the event +free again
+!wait_for_quad : free
  <- .wait("+quadrant(X1,Y1,X2,Y2)", 500);
  !!wait_for_quad.

// if I am around the upper-left corner, move to upper-right corner
+around(X1,Y1) : quadrant(X1,Y1,X2,Y2) & free
  <- !prep_around(X2,Y1).

// if I am around the bottom-right corner, move to upper-left corner
+around(X2,Y2) : quadrant(X1,Y1,X2,Y2) & free
  <- !prep_around(X1,Y1).

// if I am around the right side, move to left side two lines below
+around(X2,Y) : quadrant(X1,Y1,X2,Y2) & free
  <- ?calc_new_y(Y,Y2,YF);
  !prep_around(X1,YF).

// if I am around the left side, move to right side two lines below
+around(X1,Y) : quadrant(X1,Y1,X2,Y2) & free
  <- ?calc_new_y(Y,Y2,YF);
  !prep_around(X2,YF).

// last "around" was none of the above, goes back to my quadrant
+around(X,Y) : quadrant(X1,Y1,X2,Y2) & free & Y <= Y2 & Y >= Y1
  <- !prep_around(X1,Y).

+around(X,Y) : quadrant(X1,Y1,X2,Y2) & free & X <= X2 & X >= X1
  <- !prep_around(X,Y1).

+!prep_around(X,Y) : free <- -around(_,_); -last_dir(_); !around(X,Y).

+!around(X,Y)
  : // I am around to some location if I am near it or
    // the last action was "skip"
    // (meaning that there are no paths to there)
    (pos(AgX,AgY) & jia.neighbour(AgX,AgY,X,Y)) | last_dir(skip)
  <- +around(X,Y).

```

```

+!around(X,Y) : not around(X,Y)
  <- !next_step(X,Y);
    !!around(X,Y).
+!around(X,Y) : true
  <- !!around(X,Y).

+!next_step(X,Y)
  : pos(AgX,AgY)
  <- jia.get_direction(AgX, AgY, X, Y, D);
    -+last_dir(D);
    D.
+!next_step(X,Y) : not pos(_,_) // I still do not know my position
  <- !next_step(X,Y).
-!next_step(X,Y) : true          // failure handling, start again
  <- -+last_dir(null);
    !next_step(X,Y).

```

Implementation of Gold Handling

When a miner sees a piece of gold, three relevant plans can be considered applicable depending on its current beliefs:

```

@pcell[atomic]          // atomic: so as not to handle another
                        // event while handle gold is initialised
+cell(X,Y,gold)
  : not carrying_gold & free
  <- -free;
    +gold(X,Y);
    !init_handle(gold(X,Y)).

@pcell12[atomic]
+cell(X,Y,gold)
  : // I desire to handle another gold which
    // is farther than the one just perceived
    not carrying_gold & not free &
    .desire(handle(gold(OldX,OldY))) &
    pos(AgX,AgY) &
    jia.dist(X,Y,AgX,AgY,DNewG) &
    jia.dist(OldX,OldY,AgX,AgY,DOldG) &
    DNewG < DOldG
  <- +gold(X,Y);
    .drop_desire(handle(gold(OldX,OldY)));
    .broadcast(untell, committed_to(gold(OldX,OldY)));
    .broadcast(tell,gold(OldX,OldY));
    !init_handle(gold(X,Y)).

```

```
+cell(X,Y,gold)
: not gold(X,Y) & not committed_to(gold(X,Y))
<- +gold(X,Y);
  .print("Announcing ",gold(X,Y)," to others");
  .broadcast(tell,gold(X,Y)).
```

The first plan is applicable when the miner is not carrying gold and is free. The plan execution consists of removing the belief that it is free, adding a belief that there is gold at that location, and creating a goal to handle that particular piece of gold.

The second plan is applicable when the miner is also not carrying gold but is not free because it is going to position `OldX,OldY` to collect some gold there. In this case, it prefers the gold just found if it is closer, so the agent: (i) drops the previous intention; (ii) announces the availability of gold at the ‘old’ location to the other agents (this will trigger again the allocation protocol in Figure 9.8); and (iii) creates a goal to handle the piece of gold it has just found.

If none of the above plans is applicable (i.e. the agent is carrying gold), the third alternative plan is used to announce the gold location to others agents, starting the allocation protocol as before.

The code below implements the gold allocation protocol from the miner agent perspective (Figure 9.8). When the agent receives a message with some gold position, if it is free, it sends a bid based on its Manhattan distance to the gold; otherwise, it sends a very high bid. When some gold is allocated by the leader to an agent, that agent will intend to handle that piece of gold if the agent is still free. Note that this is not an optimal strategy: we have not as yet dealt with the possibility that reallocating tasks to the agents that are already committed (i.e. no longer free) might lead to a better overall task allocation.

```
// someone else sent me a gold location
+gold(X1,Y1)[source(A)]
: A \== self &
  not allocated(gold(X1,Y1),_) & // The gold was not allocated yet
  not carrying_gold &           // I am not carrying gold
  free &                         // and I am free
  pos(X2,Y2) & .my_name(Me)
<- jia.dist(X1,Y1,X2,Y2,D);      // bid
  .send(leader,tell,bid(gold(X1,Y1),D,Me)).

// bid high as I'm not free
+gold(X1,Y1)[source(A)]
: A \== self & .my_name(Me)
<- .send(leader,tell,bid(gold(X1,Y1),10000,Me)).

// gold allocated to me
@palloc1[atomic]
```

```

+allocated(Gold,Ag)[source(leader)]
  : .my_name(Ag) & free // I am still free
  <- -free;
  !init_handle(Gold).

// some gold was allocated to me, but I cannot
// handle it anymore, re-announce
@palloc2[atomic]
+allocated(Gold,Ag)[source(leader)]
  : .my_name(Ag) & not free // I am no longer free
  <- .broadcast(tell,Gold).

// someone else picked up the gold I am going to get,
// so drop the intention and chose another gold
@ppgd[atomic]
+picked(G)[source(A)]
  : .desire(handle(G)) | .desire(init_handle(G))
  <- .abolish(G);
  .drop_desire(handle(G));
  !!choose_gold.

// someone else picked up a gold I know about,
// remove from my belief base
+picked(gold(X,Y))
  <- -gold(X,Y)[source(_)].

```

The gold allocation code in the leader agent is:

```

+bid(Gold,D,Ag)
  : .count(bid(Gold,_,_),3) // three bids were received
  <- !allocate_miner(Gold);
  .abolish(bid(Gold,_,_)).

+!allocate_miner(Gold)
  <- .findall(op(Dist,A),bid(Gold,Dist,A),LD);
  .min(LD,op(DistCloser,Closer));
  DistCloser < 10000;
  // Allocate the gold to the closest agent
  .broadcast(tell,allocated(Gold,Closer)).

```

Finally, the plans to achieve the goal of handling a found piece of gold are shown below. The plan initially drops the goal of finding gold (exploration behaviour), moves the agent to the gold position, picks up the gold, announces to others that the gold has been collected so they do not try to fetch this gold (to avoid agents moving to pieces of gold that are no longer there), retrieves the depot location from the belief base, moves the agent to the depot, drops the gold, and finally chooses

another piece of gold to pursue. In case the handle-gold plan fails (e.g. because the gold disappeared due to the environment being dynamic), the event `-!handle(G)` is created and the second plan is selected. This plan just removes the information about that gold location from the belief base and chooses another piece of gold to be collected. The `choose_gold` plans find the nearest known gold and create a goal to handle it; if no gold location is known, the agent becomes free and resumes the gold-searching behaviour.

```
@pih1[atomic]
+!init_handle(Gold)
  : .desire(around(_, _))
  <- .drop_desire(around(_, _));
  !init_handle(Gold).

@pih2[atomic]
+!init_handle(Gold)
  : pos(X,Y)
  <- +!last_checked(X,Y);
      !!handle(Gold). // use !! so that "handle"
                      // isn't performed as atomic

+!handle(gold(X,Y))
  : not free
  <- .broadcast(tell, committed_to(gold(X,Y)));
      !pos(X,Y);
      !ensure(pick,gold(X,Y));
      .broadcast(tell,picked(gold(X,Y)));
      ?depot(_,DX,DY);
      !pos(DX,DY);
      drop;
      -gold(X,Y)[source(_)];
      !!choose_gold.

// if ensure(pick/drop) failed, pursue another gold
-!handle(G) : G
  <- -G[source(_)];
      !!choose_gold.
-!handle(_)
  <- !!choose_gold.

// no known gold to choose from...
// ... become free again to search for gold
+!choose_gold
  : not gold(_,_)
  <- +!free.
```

```

// Finished one gold, but others left
// find the closest gold among the known options
// that nobody else committed to
+!choose_gold
  : gold(_,_)
  <- .findall(gold(X,Y),gold(X,Y),LG);
    !calc_gold_distance(LG,LD);
    .length(LD,LLD); LLD > 0;
    .print("Uncommitted gold distances: ",LD,LLD);
    .min(LD,d(D,NewG));
    .print("Next gold is ",NewG);
    !!handle(NewG).
-!choose_gold
  <- --free.

+!calc_gold_distance([],[]).
+!calc_gold_distance([gold(GX,GY)|R],[d(D,gold(GX,GY))|RD])
  : pos(IX,IY) & not committed_to(gold(GX,GY))
  <- jia.dist(IX,IY,GX,GY,D);
    !calc_gold_distance(R,RD).
+!calc_gold_distance([_|R],RD)
  <- !calc_gold_distance(R,RD).

+!ensure(pick,G)
  : pos(X,Y) & cell(X,Y,gold)
  <- pick;
    ?carrying_gold.
    // fails if there's no gold there or not carrying_gold after
    // pick!
    // handle(G) will "catch" this failure.

+!pos(X,Y) : pos(X,Y).
+!pos(X,Y) : not pos(X,Y)
  <- !next_step(X,Y);
    !pos(X,Y).

```

It is important to note that AgentSpeak is the language used to define the high-level (practical) reasoning of the agents. The AgentSpeak code for the team of gold miners, in our opinion, is a quite elegant solution, being declarative, goal-based (i.e. based on the BDI architecture), and also neatly allowing agents to have long-term goals while reacting to changes in the environment. Some of the advantages of using *Jason* are the support for high-level (speech-act based) communication, transparent integration with the contest server and support to the use of existing Java code through internal actions (e.g. for the A* algorithm). Although not a ‘purely’ declarative, logic-based approach, the combination of both declarative and

legacy code allowed for an efficient solution without compromising the high-level declarative part of the implementation.

In future versions of this team, we plan to avoid the use of centralised negotiation (which has the leader as a single point of failure) and to use *Moise*⁺ [56] to create an organisation with the specification of the roles in the system (we say more about *Moise*⁺ in Section 11.2). In our original strategy, there was yet another role which was that of the ‘courier’; where the depot happens to be in a position too far from some of the quadrants, the courier will help carry to the depot pieces of gold from agents that are in the more distant quadrants; this is yet to be implemented. We also plan to experiment with DCOP algorithms for optimal allocation of agents to collect pieces of gold.

9.2 Case Study II: Electronic Bookstore

As mentioned in the introduction of this chapter, the electronic bookstore application is the example used throughout the book *Developing Intelligent Agent Systems* [75]. In this section, we do not describe the implementation of the whole system, since we are particularly interested in illustrating the integration of *Jason* with various technologies.

The main goal of this application is to sell books in an online system. To achieve this goal, among all identified functionalities available in the specification presented in [75], we selected the following to give as implementation examples:

Online interaction: manages the interaction with a single user, via the website.

Book finding: locates book information according to a given description.

Purchasing: manages online book sales.

Stock management: keeps track of the available stock.

These functionalities are distributed among three agents: sales assistant (online interaction, purchasing), delivery manager (purchasing) and stock manager (stock management, book finding). We also add an additional agent to simplify the implementation: the login agent. Based on a database of registered users, it validates the login of an user in the system and, if approved, creates the corresponding sales assistant for that user. Thus, each user logged on to the system has one sales assistant agent for her/himself.

All the above agents are developed in AgentSpeak and therefore can interact using *Jason*’s support for communication. However, the web application is not a *Jason* agent, and yet has to interact with the other agents. These agents may also be distributed in several different network hosts. Therefore, the centralised infrastructure cannot be used in this case; the Saci infrastructure was selected to

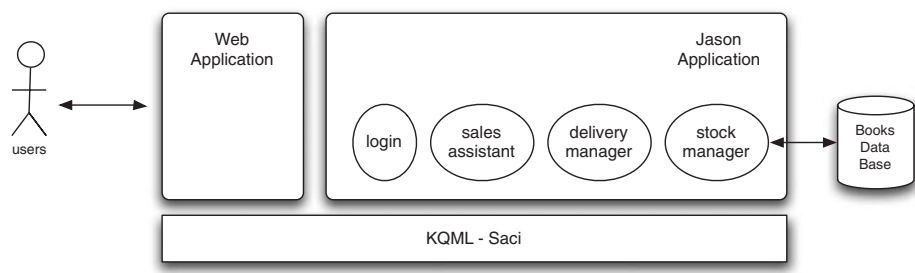


Figure 9.9 Overall architecture of the bookstore application.

provide distribution and (KQML-based) communication among heterogeneous agents. Figure 9.9 shows the overall architecture of the system. The next two sections describe each part – the web application and *Jason* agents – of this architecture.

Web Application Agent

The whole web application was developed with Java Server Pages (JSP) and thus runs on an application server. An example of a search result in this web site is shown in the screenshot in Figure 9.10. To allow this application to send and receive KQML-like messages, the application has a Java bean component (a Saci mailbox) created when the application starts and shared by all pages.

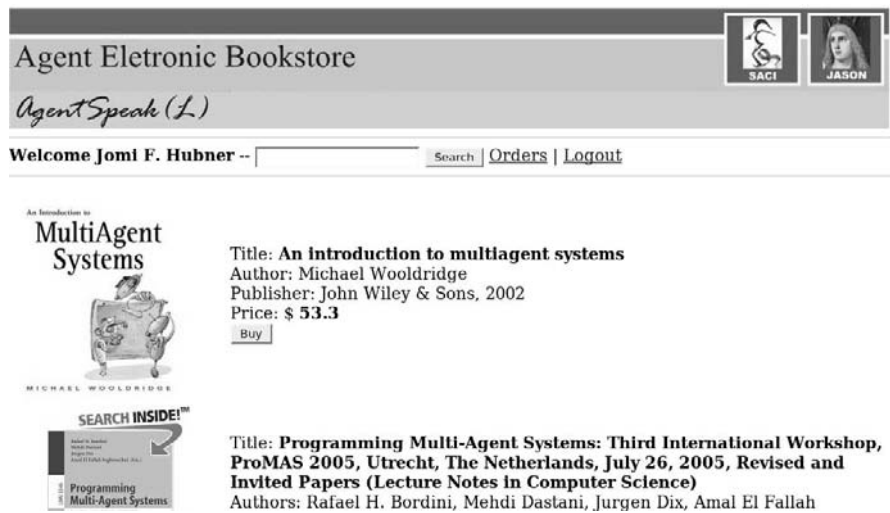


Figure 9.10 Electronic bookstore example web page.

For example, the `login.jsp` page receives the user name and password filled in by the user in a form, sends a message to the login agent to validate the user, and if it is valid forwards the user to its personal page. Part of the code for this page is:

```
...
// Declaration of the Saci mail box bean.
<jsp:useBean id="mbox" scope="application"
              class="saci.MBoxSAg" />
...

// get parameters from the form
String userName = request.getParameter("txtUser");
String password = request.getParameter("txtPassword");

// creates a KQML message to send to the 'login' agent
Message m =
    new Message("(askOne :receiver login :language AgentSpeak)");
m.put("content", "user_logon("+userName+",\""+password+"\")");

// sends a message and waits for the answer
Message r = mbox.ask(m);

String content = r.get("content").toString();
if (content.startsWith("client")) {
    Structure s = Structure.parse(content);
    String name = s.getTerm(0).toString();
    session.setAttribute("username", userName);
    session.setAttribute("name", name);
    %>
    <jsp:forward page="userPage.jsp" />
    <%
}
...
```

To illustrate the communication between the web application (more precisely, its `login.jsp` page) and the login agent, let us suppose that the user logging in to the systems is 'bob' with password 'test'; the message sent to the login agent will be:

```
(askOne :sender      web
        :receiver    login
        :reply-with  id1
        :language    AgentSpeak
        :content      "user_logon(bob,\"test\")")
```

The sender and reply-with slots are automatically added by the ask method of the mailbox. This method also waits for the answer which, if the password is correct, is:

```
(tell      :sender      login
      :receiver    web
      :in-reply-to id1
      :language    AgentSpeak
      :content     "client(\"Bob Spengler\")")
```

The web page finally registers the user name and full name in the user's session so that other pages can access this information, and forwards it to the user's personal page. The remaining pages of the application do not send messages to the login agent but to the sales assistant created for the user (there is a guest sales assistant for users not logged in), as in the following excerpt of the search page:

```
<jsp:useBean id="mbox" scope="application" class="saci.MBoxSAG" />

<%
String key = request.getParameter("key");

if (key != null && key.length() > 0) {
    String userName = session.getAttribute("username").toString();

    Message m = new Message("askOne");
    m.put("receiver", userName);
    m.put("content", "find_book(\""+key+"\",L)");
    Message answer = mbox.ask(m);

    // the answer is a list of books code
    Literal content = Literal.parseLiteral(
        answer.get("content").toString());
    ListTerm lbooks = (ListTerm)content.getTerm(1);
    ...
}
```

In the search protocol, the message sent by the web application is

```
(askOne :sender      web
      :receiver    bob
      :reply-with id2
      :language    AgentSpeak
      :content     "find_book(\"Agent\",L)")
```

and the answer is something like

```
(tell      :sender      bob
      :receiver    web
```

```

:in-reply-to id2
:language      AgentSpeak
:content       "find_book(\"Agent\", [3,4,30])"

```

where [3,4,30] is a list of book references for which the keyword ‘Agent’ was found.

Jason Agents

While the web application focuses on the interface, the functionalities of the system are implemented as agents in *Jason*. The system starts with four agents: the three agents described earlier (and defined in [75]) and a default sales assistant, called *guest*, for guest users (i.e. those who are not logged on to the system). More sales assistant agents are dynamically created when users log in. These agents use *Saci* as infrastructure and so can be distributed in different hosts as defined in the following system configuration:

```

MAS electronicBookstore {
  infrastructure:
    Saci

  agents:
    login
      beliefBaseClass jason.bb.TextPersistentBB
      at "web.server.com"; // the host where this agent starts

    guest salesAssistant at "web.server.com";

    deliveryManager
      beliefBaseClass jason.bb.TextPersistentBB
      at "int.server.com";

    stockManager
      beliefBaseClass jason.bb.JDBCPersistentBB(
        "org.hsqldb.jdbcDriver", // driver for HSQLDB
        "jdbc:hsqldb:bookstore", // URL connection
        "sa", // user
        "", // password
        "[book(7),publisher(2),book_stock(2),
          author(2),book_author(2),
          topic(2),book_topic(2)]"
      )
      at "bd.server.com";
}

```

In this configuration, text files are used to persist the belief base of the login and delivery manager agents. The stock manager uses a database to persist and

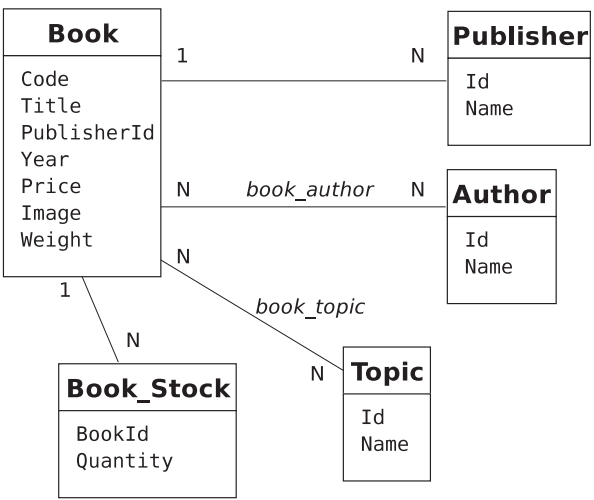


Figure 9.11 Entity-relationship model of the stock manager database.

access the information about book title, publisher, author, topic and the relations between them (the features described in Section 7.4 are used to bind the beliefs to tables of a relational database). The entity-relationship model of this database is specified in Figure 9.11. We will not include here all the AgentSpeak source code of the agents, since most of them are quite simple and are available at the web page for this book (<http://jason.sf.net/jBook>). Below, we comment on parts of the code for the stock manager and login agents.

The stock manager has access to the book database (modelled as in Figure 9.11) and thus is responsible for finding a book given a keyword, for instance. When it receives a message, with content `find_book(Key,BookList)`, from the web application asking for a search, the default implementation of KQML tries to find a belief that unifies with the content (see Chapter 6 for more information on communication). Of course, there is no such belief. In this case, an event `+?find_book(Key,BookList)` is generated and handled by the last plan in the code below. Note that this last plan makes extensive use of rules and backtracking to implement the ‘find book’ functionality of this agent.

```

// find books by some author
//  the first argument is the author to be found and
//  the second is a book's code
find_by_author(SearchAuthor,Code)
:-  book(Code, _, _, _, _, _, _) & // for all books in DB
    book_author(Code,AuthorId) &   // for all its authors
    author(AuthorId,Name) &        // get author's name
    .substring(SearchAuthor,Name) .
    
```

```
// find book by its title
find_by_title(SearchTitle,Code)
  :- book(Code, Title, _, _, _, _ &
      .substring(SearchTitle,Title).

// the plan for askOne find_book
+?find_book(Key, L)
  <- .findall(CA,find_by_author(Key,CA),LA);
      .findall(CT,find_by_title(Key,CT),LT);
      .concat(LA,LT,L).
```

The login agent implements the login protocol described in the introduction of this case study. It has to create a new agent for each user that logs in to the system, and has to kill this agent when the user logs out.

```
+!kqml_received(S, askOne, user_logon(Username, Password), M)
  : // S is the sender and M is the reply-with id
      client(Username, Password, Name, _, _, _, _, _ &
  <- // creates an agent for this user
      .create_agent(
          Username, // agent name
          "salesAssistant.asl", // source code
          [beliefBaseClass(jason.bb.TextPersistentBB)]); // options
      .send(S, tell, client(Name), M).

+!kqml_received(S, askOne, user_logon(Username, Password), M)
  <- .send(S, tell, error("Username or password invalid!"), M).

+user_logout(Username) [source(web)]
  <- .kill_agent(Username).
```

The first two plans exemplify customisations of the *Jason* default communication semantics. The default semantics for the askOne is not suitable for the desired login protocol, where the answer (client(Name)) does not unify the ask content (user_logon(Username,Password)). Of course, the login protocol could be changed to fit the *Jason* usual features. However, the intention here is to demonstrate how a KQML-compliant agent that was not implemented in *Jason* can interact with a *Jason* agent. With the customisation, when an askOne user_logon message arrives, these two plans are relevant. The first is applied when the user and password are correct, according to a belief client(...) that is used to represent all valid users; the second plan is used otherwise. To dynamically create a new sales assistant agent, the internal action .create_agent is used (see page 252 for more detail on this internal action and its arguments). Similarly, the internal action .kill_agent is used in the last plan to kill the sales assistant agent when the user logs out.